Differentiable Lattice Boltzmann Simulation with Second-Order Boundary Conditions for Gradient-Based Shape Optimisation

Marcos Ashton Iglesias

Department of Computer Science, University of Exeter, Exeter, UK

Abstract

We present a fully differentiable D3Q19 Lattice Boltzmann solver in JAX that supports gradient-based shape optimisation through second-order Bouzidi interpolated bounce-back boundary conditions. The Bouzidi fractional wall distances are computed via differentiable ray-surface intersection, providing a gradient path from obstacle geometry through the boundary treatment. This is the enabling difference: in the standard LBM formulation (halfway bounce-back with a binary solid mask) the drag is a piecewise-constant function of any smooth shape parameter, so the shape gradient is exactly zero almost everywhere and gradient-based optimisation is impossible without additional machinery. We verify directly that $\partial C_d/\partial r = 0$ in that setting and non-zero under the Bouzidi pipeline. With a hard solid mask and Bouzidi q, Adam drives a physically plausible baseline ($C_d = 1.77$) down by 5.8% via radius shrinkage. Adding a sigmoid-smoothed solid representation provides an additional gradient path; we report its benefits and its pitfalls (degenerate optima that exploit the smoothing) and show that the resulting behaviour depends strongly on the sharpness parameter. Solver validation is via Poiseuille flow and a four-resolution sphere drag convergence study against the Clift, Grace, and Weber (1978) correlation at $Re = 100$, with time-averaged C_d converging to within 7%. Local gradient correctness is verified against finite differences in float64 to relative errors below 10^{-9}. The solver, including BGK and MRT collision operators, is open-source at `https://github.com/MarcosAsh/LBM_JAX_Autodiff`.

Keywords: lattice Boltzmann method, automatic differentiation, shape optimisation, Bouzidi bounce-back, JAX, inverse design

1 Introduction

The Lattice Boltzmann Method (LBM) has become a standard tool for computational fluid dynamics, particularly for flows around complex geometries where its regular grid structure and local update rules give it an advantage over unstructured mesh methods [1]. A natural next step is to make LBM simulations differentiable, so that gradients of flow quantities with respect to design parameters can be computed efficiently. These gradients enable gradient-based optimisation of boundary conditions, material properties, and obstacle geometry without resorting to finite-difference sweeps or evolutionary methods.

Adjoint-based sensitivity analysis for LBM has been explored by several groups. Tekitek et al. [2] derived adjoint equations for LBM and applied them to parameter identification. Krause et al. [3] extended this to shape optimisation within the OpenLB framework. Norgaard et al. [4] applied topology optimisation to unsteady LBM flows, demonstrating that LBM's regular grid structure is well suited to density-based design methods.

More recently, automatic differentiation (AD) through fluid simulations has gained traction. PhiFlow [5] provides a differentiable Navier-Stokes solver in a deep-learning framework. Within the LBM community, Lettuce [6] implements a differentiable lattice Boltzmann solver in PyTorch, and XLB [7] provides a massively parallel JAX-based LBM framework with AD support. These tools exploit reverse-mode AD without requiring manual adjoint derivation. Established non-differentiable LBM frameworks such as Palabos [8] remain widely used but do not natively support gradient computation through the simulation.

A critical gap in the existing literature concerns boundary treatment. Most differentiable LBM implementations use standard halfway bounce-back, which places the effective wall exactly at the midpoint between fluid and solid nodes. This is first-order accurate and, more importantly for optimisation, provides no useful gradient with respect to wall position: the wall location is locked to the grid, so the gradient of any flow quantity with respect to geometry is zero almost everywhere (it changes only when a cell flips between solid and fluid).

Bouzidi interpolated bounce-back [9] resolves this by using the fractional distance $q \in [0, 1]$ from the fluid

node to the wall along each lattice link. The streaming step interpolates the bounced distribution using q, giving second-order wall accuracy on curved surfaces. The key observation is that q is a continuous, differentiable function of the obstacle geometry. For a sphere of radius r, the ray-sphere intersection formula gives q as a smooth function of r. Reverse-mode AD traces through this formula automatically, delivering $\partial C_d/\partial r$ through the chain

$$\frac{\partial C_d}{\partial r} = \sum_{\text{links}} \frac{\partial C_d}{\partial f_i} \cdot \frac{\partial f_i}{\partial q} \cdot \frac{\partial q}{\partial r}. \tag{1}$$

In this paper we present a D3Q19 LBM solver implemented entirely in JAX [16] that makes this gradient chain concrete. Every component of the solver, from collision to streaming to force computation, is a pure function compatible with `jax.grad`. We validate the solver against the Poiseuille analytic solution and against a four-resolution sphere-drag convergence study at $Re = 100$, and demonstrate gradient-based shape optimisation under four boundary treatments that isolate the role of each differentiable path. The main comparison we draw is not with a weaker variant of the Bouzidi pipeline: it is with classical LBM itself (halfway bounce-back + binary solid mask), for which we show the shape gradient is exactly zero. Making it non-zero is the enabling contribution of this work.

2 Methodology

2.1 D3Q19 lattice Boltzmann

We use the three-dimensional D3Q19 lattice with 19 discrete velocity directions: one rest, six axis-aligned, and twelve diagonal. The quadrature weights are $w_0 = 1/3$, $w_{1\text{–}6} = 1/18$, and $w_{7\text{–}18} = 1/36$. The lattice speed of sound squared is $c_s^2 = 1/3$.

The equilibrium distribution at each node is

$$f_i^{\text{eq}} = w_i \rho \left(1 + \frac{\mathbf{e}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{e}_i \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u} \cdot \mathbf{u}}{2c_s^2}\right), \tag{2}$$

where $\rho = \sum_i f_i$ is the macroscopic density and $\rho\mathbf{u} = \sum_i f_i \mathbf{e}_i$ is the momentum.

2.2 Collision operators

2.2.1 BGK (single relaxation time)

The Bhatnagar-Gross-Krook operator [10] relaxes each distribution toward equilibrium at a uniform rate:

$$f_i^{\text{out}} = f_i - \frac{1}{\tau}(f_i - f_i^{\text{eq}}), \tag{3}$$

where τ is the relaxation time. The kinematic viscosity is $\nu = c_s^2(\tau - 1/2) = (\tau - 1/2)/3$.

2.2.2 MRT (multiple relaxation time)

The MRT operator [11] transforms to moment space via a matrix M, relaxes each moment independently, and transforms back:

$$\mathbf{f}^{\text{out}} = \mathbf{f} - M^{-1} S (M\mathbf{f} - M\mathbf{f}^{\text{eq}}), \tag{4}$$

where $S = \text{diag}(s_0, \ldots, s_{18})$ contains the per-moment relaxation rates. Conserved moments (density, momentum) have $s_k = 0$. Stress moments relax at the physical rate $s_k = 1/\tau$. Ghost and higher-order moments are damped aggressively ($s_k = 1.2$–1.98) following [11]. Our implementation uses the sparse forward and inverse transforms, avoiding a full 19×19 matrix multiply.

2.3 Bouzidi interpolated bounce-back

At solid boundaries, distributions that would stream from inside the wall are replaced by bounced values. The Bouzidi scheme [9] uses the fractional distance q from the fluid node to the wall along the lattice link:

$$f_i = \begin{cases} \frac{1}{2q} f_{i'}^* + \left(1 - \frac{1}{2q}\right) f_i^* & q \geq 0.5, \\ 2q\, f_{i'}^* + (1 - 2q)\, f_{i'}^*(\mathbf{x} + \mathbf{e}_i) & q < 0.5, \end{cases} \tag{5}$$

where i' is the opposite direction and f^* denotes post-collision values. The $q < 0.5$ branch reaches one cell further into the fluid (the neighbour at $\mathbf{x} + \mathbf{e}_i$), which widens the computational stencil and must be handled carefully at domain boundaries and in parallel decompositions.

For a sphere of radius r centred at $\mathbf{c}$, the wall distance along a lattice link from fluid node $\mathbf{x}$ in direction $\mathbf{e}_i$ is obtained by solving the ray-sphere intersection:

$$q = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \tag{6}$$

with $a = |\mathbf{d}|^2$, $b = 2(\mathbf{x} - \mathbf{c}) \cdot \mathbf{d}$, $c = |\mathbf{x} - \mathbf{c}|^2 - r^2$, where $\mathbf{d}$ is the lattice velocity vector in world coordinates. This is a smooth function of r, and JAX's reverse-mode AD traces through it automatically.

2.4 Differentiable solid representation

The binary solid mask is a step function with zero gradient almost everywhere. Following List et al. [15], we replace it with a sigmoid-smoothed porosity field:

$$\phi(\mathbf{x}) = \sigma(\alpha(r - |\mathbf{x} - \mathbf{c}|)), \tag{7}$$

where σ is the logistic sigmoid and α controls the transition sharpness. This is a continuous volume penalisation: in the collision step, each cell blends between fluid and solid behaviour according to its porosity:

$$f_i^{\text{out}} = (1 - \phi) f_i^{\text{collided}} + \phi\, f_{i'}^{\text{collided}}. \tag{8}$$

Cells deep inside the obstacle ($\phi \approx 1$) undergo full bounce-back; cells far from the surface ($\phi \approx 0$) collide normally; and a thin transition zone near the surface provides the smooth gradient bridge. This volume penalisation acts on the collision step, complementing the Bouzidi gradient path which acts on the streaming step. Together, they give the optimiser gradient signal through both components of the LBM update.

2.5 Force computation

We use the Mei-Luo-Shyy momentum exchange method [13]. For each fluid node adjacent to a solid boundary, the force contribution from lattice link i is $\mathbf{F}_i = \mathbf{e}_i(f_i^* + f_{i'})$. The drag coefficient is

$$C_d = \frac{|F_x|}{\frac{1}{2}\rho_\infty U_\infty^2 A}, \tag{9}$$

where A is the projected frontal area.

2.6 Boundary conditions

At the inlet ($x = 0$) we apply Zou-He velocity boundary conditions [14]. At the outlet ($x = N_x - 1$) we apply Zou-He with prescribed pressure ($\rho = 1$). The y and z boundaries are periodic.

2.7 Gradient checkpointing

The simulation is compiled into a single XLA program via `jax.lax.scan`. Each step is wrapped with `jax.checkpoint`, which recomputes forward values during the backward pass instead of storing them. Memory drops from $O(N)$ intermediate states to $O(1)$.

3 Validation

3.1 Poiseuille flow

We simulate pressure-driven flow between two parallel walls separated by $H = 18$ lattice cells in a $60 \times 20 \times 4$ domain. The inlet velocity is $U = 0.05$, the relaxation time is $\tau = 0.8$ ($\nu = 0.1$), and we run 3000 BGK steps.

Figure 1 shows the velocity profile at the channel midpoint compared to the analytic parabolic solution. Table 1 gives representative numerical values. The relative L2 error over the fluid cells is 3.8×10^{-3} (0.38%). The profile is symmetric to machine precision.

3.2 Sphere drag at $Re = 100$

We validate the external-flow pipeline on the canonical test case of uniform flow past an isolated sphere at $Re = 100$. The reference value from the Clift, Grace, and Weber [12] drag correlation is $C_d = 1.09$. In every case below the sphere has radius 0.5 in world coordinates, is centred in a 2:1:1 domain, and sees free-stream $U = 0.05$ in lattice units, with τ set by $\nu = UD/Re$. Boundaries are Zou-He inlet/outlet and periodic in y, z.

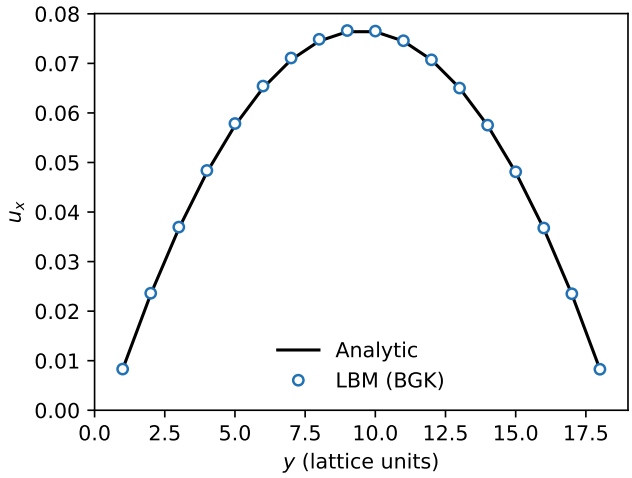

Figure 1: Poiseuille velocity profile at the channel midpoint ($x = 30$). Open circles: LBM simulation (3000 BGK steps). Solid line: analytic parabolic solution. The relative L2 error is 0.38%.

Table 1: Poiseuille flow: LBM vs analytic velocity. $H = 18$, $\tau = 0.8$, 3000 steps.

y	u_x (LBM)	u_x (analytic)
0	0.0002	0.0000
1	0.0082	0.0083
5	0.0579	0.0577
9	0.0769	0.0766
10	0.0769	0.0766
15	0.0484	0.0482
19	0.0002	0.0000

Two observations turned out to matter more than expected. First, the wake in this regime has low-frequency unsteadiness: a single-snapshot C_d moves by 10–15% between samples. Any meaningful validation must time-average. Second, the blockage ratio $D/L_y = 0.25$ that we use throughout is known to inflate measured C_d by several percent [12, ch. 9]; the finite-domain correction is physics, not numerical error. We chose not to widen the domain because the paper's focus is the gradient pipeline, not a quantitatively tight C_d match.

Table 2 reports a four-resolution grid convergence study. Step counts scale with n_x so every grid sees at least five flow-through times; the reported C_d is the mean of samples taken every 500 steps over the final 30% of the run, and the reported σ is their standard deviation (a quantitative fingerprint of the wake unsteadiness). MRT ghost-mode relaxation rates are held at d'Humières' default damping values [11]: we did not tune them to the target Re, which is a known source of additional a few-

Table 2: Sphere C_d grid convergence at $Re = 100$ (MRT + Bouzidi). Time-averaged over the final 30% of the run.

Grid	D	steps	$C_d \pm \sigma$	|err|
64× 32× 32	8	8,000	1.29 ± 0.02	18.4%
128× 64× 64	16	16,000	1.19 ± 0.04	8.8%
256×128×128	32	28,000	1.19 ± 0.07	8.8%
384×192×192	48	40,000	1.17 ± 0.08	7.1%

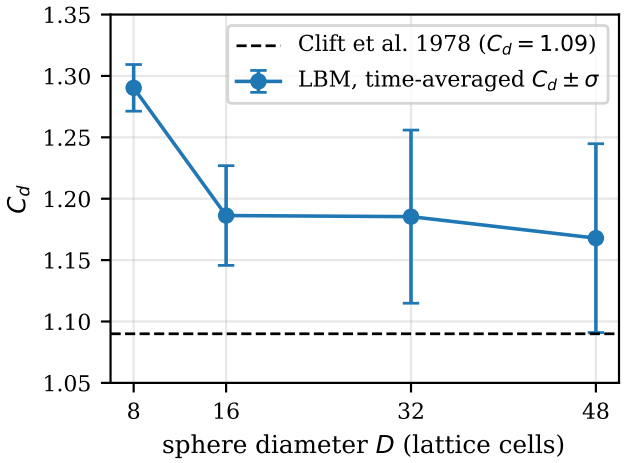

Figure 2: Sphere C_d grid convergence at $Re = 100$. Points: LBM time-averaged C_d with $\pm\sigma$ error bars (Table 2). Dashed line: Clift correlation. The residual ∼7% gap at the finest grid is consistent with blockage ($D/L_y = 0.25$) and the un-tuned MRT relaxation rates.

percent error [13] and a deliberate trade-off to keep the experiment reproducible rather than parameter-swept.

The measured error decreases monotonically with refinement and settles near 7% at the finest grid, where the residual gap is consistent with the blockage correction and the un-tuned MRT parameters. The observed convergence is not clean first- or second-order: the sequence hits a plateau because the remaining error is physics (blockage, reference correlation uncertainty, wake unsteadiness captured by finite-time averaging) rather than truncation. We report this explicitly rather than computing a nominal convergence order that would be dominated by those sources.

For comparison, BGK on the $128 \times 64 \times 64$ grid at the same $\tau = 0.524$ produces $C_d = 0.75$ with rising velocity magnitude and large oscillations, confirming that MRT is the right choice near the lower limit of τ.

3.3 Gradient verification

Gradient correctness is verified at two scales.

Table 3: Local gradient verification in float64, short simulations.

Test	AD	FD	Rel. err
$\partial(\sum f^{eq})/\partial\rho$	64.000001	63.999997	6.2×10^{-8}
$\partial\bar{\rho}/\partial U$ (5 steps)	0.611238	0.611238	1.9×10^{-9}
Bouzidi $\partial C_d/\partial r$ (10 steps)	93.33275	93.33275	9.0×10^{-10}

Table 4: Grid convergence of $\partial C_d/\partial r$ under the paper's differentiable setup (MRT, soft porosity, Bouzidi-q, radius $r = 0.5$, $Re = 50$, $\epsilon = 10^{-3}$).

Grid	D	steps	AD	FD	Rel. err
$32 \times 16 \times 16$	4	1920	−7.21	−5.75	25%
$48 \times 24 \times 24$	6	2880	+6.10	+6.69	9%
$64 \times 32 \times 32$	8	3840	+7.10	+5.17	37%
$96 \times 48 \times 48$	12	5760	+1.05	+1.35	22%

3.3.1 Local correctness (10 steps, float64)

We first confirm that reverse-mode AD through equilibrium, streaming, and Bouzidi produces the mathematically correct derivative, by running short simulations in float64 mode (`JAX_ENABLE_X64`) on CPU and comparing against central finite differences with step size $\epsilon = 10^{-4}$. Table 3 reports three representative cases.

All three agree to better than 10^{-7} relative error, confirming that AD is implemented correctly locally. The verification is on CPU in float64; the production GPU pipeline runs float32.

3.3.2 Long-horizon behaviour across resolutions

Local correctness is necessary but not sufficient. In the shape optimisation pipeline the gradient is taken through thousands of simulation steps in float32, a regime in which differentiable fluid solvers are known to accumulate sensitivities and lose numerical agreement with FD even when AD is implemented exactly. We therefore also run a grid-convergence study of $\partial C_d/\partial r$ at four resolutions ($D = 4, 6, 8, 12$) using the paper's soft-porosity and Bouzidi-q differentiable pipeline, MRT collision, and ∼3 flow-through times per run. The backward pass uses $O(\sqrt{n_{\text{steps}}})$ activation memory via chunked checkpointing. Results are in Table 4.

The coarsest grid ($D = 4$, four cells across the sphere) reports a negative gradient: unphysical, but AD and FD agree on the (wrong-signed) answer, i.e. the solver is faithfully differentiating the geometry it actually sees at that resolution rather than the sphere one would want. Below about $D = 6$, Bouzidi is outside its accuracy regime and the gradient should not be trusted. From $D = 6$ upward, AD and FD agree on sign; their relative disagreement is 9–37% and reflects the combined effect of float32 precision, wake unsteadiness, and the sigmoid-smoothed geometry whose derivative is the dominant

contributor to the gradient magnitude (see Section 4.4). We view these numbers as the realistic accuracy band of the production gradient, and we frame the optimisation results accordingly: Adam is robust to this level of gradient noise, but no part of the paper treats the gradient as a high-precision quantity at long horizons.

4 Shape optimisation

4.1 Problem setup

The objective is $\min_r C_d(r)$ for a sphere of radius r in uniform flow at $Re = 50$. Each iteration runs a differentiable forward simulation on a $96 \times 48 \times 48$ grid ($D = 12$) with MRT collision, 2000 steps ($\tau = 0.536$, $\sim$2 flow-through times), Zou-He inlet/outlet, and periodic y, z. Adam updates r with $lr = 3 \times 10^{-3}$ for 60 iterations starting from $r = 0.5$, with r clamped to $[0.3, 0.8]$ (the lower bound keeps the sphere in the regime where Bouzidi resolves its curvature; see Section 3.3.2).

This setup is deliberately larger and longer than an earlier 48x24x24, 150-step setup we tried: that grid produced a non-physical baseline $C_d \approx 0.43$ at which the forward flow had not yet developed, which in turn made any optimisation result a property of the transient rather than of the geometry. The $96 \times 48 \times 48$ setup gives a physical baseline in the case that matters most (Section 4.3) and enables the boundary-treatment comparison below.

4.2 Four boundary treatments

To separate the contributions of the two differentiable paths in the pipeline (the sigmoid porosity of Equation (7) on the collision step, and the Bouzidi q on the streaming step), we run the same optimisation problem under four boundary treatments:

- **halfway_hard**: binary solid mask, $q = 0.5$ everywhere. This is classical production LBM.
- **bouzidi_hard**: binary solid mask, q from ray-sphere intersection (Equation (6)).
- **halfway_soft**: sigmoid porosity ($\alpha = 20$), $q = 0.5$ everywhere.
- **bouzidi_soft**: sigmoid porosity, q from ray-sphere intersection. (This is the full pipeline of the paper.)

Table 5 reports baseline C_d, best C_d reached, mean $|\partial C_d/\partial r|$, and the final radius for each.

4.3 Classical LBM has no shape gradient

The **halfway_hard** row is the enabling observation of this paper. With a binary solid mask and halfway bounce-back, the solver produces a physically reasonable $C_d = 1.85$ at $r = 0.5$ (an expected few-percent overestimate of the Clift $Re = 50$ value, consistent with the blockage and un-tuned MRT discussion in Section 3.2). Its derivative with respect to r is exactly 0.0: AD returns zero, not a small numerical residual, because the only place r enters the computational graph is through the solid mask ($|\mathbf{x} - \mathbf{c}| \le r$) which is a step function. There is no differentiable path from geometry to drag, so gradient-based optimisation is impossible in this setting without additional machinery.

Table 5: Four-mode shape optimisation on the $96 \times 48 \times 48$ grid. Adam, 60 iterations, $r_0 = 0.5$.

Mode	base C_d	best C_d	avg $\lvert g\rvert$	final r
halfway_hard	1.85	–	0.0	0.500
bouzidi_hard	1.77	1.66	5.55	0.314
halfway_soft	0.43	0.003	6.30	0.652
bouzidi_soft	0.45	0.002	3.19	0.663

With Bouzidi q and the same hard mask (**bouzidi_hard**), the gradient becomes non-zero (5.55 on average) and Adam drives the optimisation from $C_d = 1.77$ at $r = 0.5$ to $C_d = 1.66$ at $r = 0.31$ over 60 iterations, a 5.8% reduction by radius shrinkage. This is the clean result: the only differentiable path is through $q(r)$, the optimiser uses it, and the outcome is physically sensible (a smaller sphere has less drag).

The two rows together are the core claim: the existence of an AD path through Bouzidi is what makes gradient-based shape optimisation possible at all within the LBM, independent of any further smoothing of the geometry.

4.4 Soft porosity: additional signal, degenerate optima

Adding the sigmoid porosity field of Equation (7) introduces a second gradient path through the collision step. Both **halfway_soft** and **bouzidi_soft** have this path, and in both modes the optimiser drives C_d close to zero while *growing* the radius to roughly 0.66.

This is not an aerodynamic result. At $r = 0.66$ the soft obstacle fills most of the $L_y = L_z = 4$ cross-section, and the force on the obstacle stays small while the projected area A in the C_d denominator grows. The optimiser exploits the smoothing: nothing penalises it for expanding a transparent blob that plugs the channel, and the porosity gradient points in that direction. This is a design-space pathology, not a solver defect: any objective that normalises by A and any soft-mask parameterisation will admit it unless the geometry or the loss is constrained.

Two further observations follow from this mode:

1. The *gradient magnitude* comparison between **bouzidi_soft** and **halfway_soft** does not favour Bouzidi. Their avg $|\partial C_d/\partial r|$ values are 3.19 and 6.30 respectively; the porosity path dominates and is equally available in both. The often-cited "stronger gradients under Bouzidi" effect arises only when the porosity path is absent, which in our

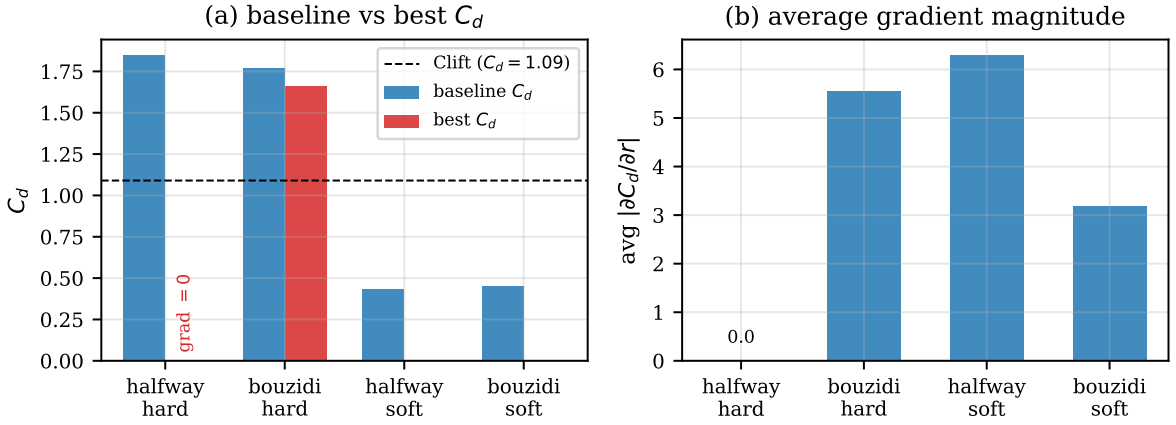

Figure 3: Four-mode optimisation summary from Table 5. **(a)** Baseline vs best C_d per mode. `halfway_hard` cannot be optimised: its shape gradient is exactly zero. `bouzidi_hard` starts from a physical baseline ($C_d = 1.77$, close to the Clift dashed line) and reaches 1.66 by shrinking the radius, the clean physical result. Both soft-porosity modes drive $C_d \to 0$ via the degenerate "grow to plug the channel" solution discussed in Section 4.4. **(b)** Average $|\partial C_d/\partial r|$ over 60 iterations. The classical halfway-hard bar is exactly zero (grey); every other mode has a non-zero gradient, but the porosity path dominates the magnitude in both soft modes, so the common "Bouzidi gives stronger gradients" comparison is mostly a statement about soft porosity rather than about Bouzidi itself.

Table 6: Sensitivity of C_d and $\partial C_d/\partial r$ to the sigmoid sharpness α, on the $96 \times 48 \times 48$ grid at $r = 0.5$, 2880 MRT steps.

α	C_d	AD	FD	Rel. err
5	3.76	+15.16	+15.47	2.0%
10	1.35	+ 3.66	+ 3.70	1.0%
20	0.87	+ 1.85	+ 2.28	18.8%
40	0.33	+ 9.49	+10.40	8.8%
80	0.54	+ 2.44	+ 3.35	27.2%

experiments is the hard-mask case where halfway bounce-back gives zero and Bouzidi gives 5.55.

2. The baseline C_d under soft porosity (0.43, 0.45) is well below the hard-mask value (1.85). The sigmoid smooths the wall so effectively that the obstacle bleeds momentum rather than imposing no-slip: at $\alpha = 20$ the transition zone is roughly $1/\alpha = 0.05$ world units wide, which on this grid is 1–2 cells. That is thin enough to look sharp on a figure but thick enough to change C_d by a factor of four.

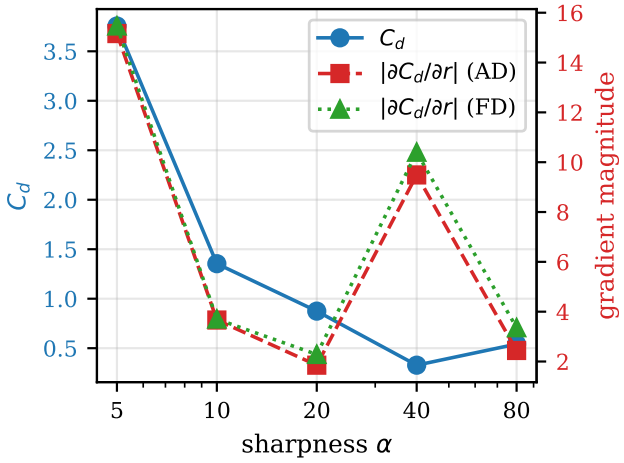

Figure 4: Sensitivity of C_d (blue, left axis) and $|\partial C_d/\partial r|$ (red/green, right axis) to the sigmoid sharpness α at $r = 0.5$ on the $96 \times 48 \times 48$ grid. The gradient sign is stable across a decade in α; magnitudes are not. AD and FD track each other closely throughout.

4.5 Sharpness sensitivity

Given the above, the sigmoid sharpness α is a consequential hyperparameter. Table 6 reports C_d and $\partial C_d/\partial r$ at fixed $r = 0.5$ on the same grid, as α varies over a decade.

Both C_d and the gradient magnitude depend strongly on α: C_d spans an order of magnitude, and |grad| spans roughly the same. The gradient sign, however, is stable (positive at every α). AD and FD agree within 3% at the small-α smooth end where the sigmoid is effectively linear over the ϵ-perturbation, and the disagreement widens to 27% as α grows and the transition localises. Our choice of $\alpha = 20$ is not cherry-picked for the optimisation outcome, but it is a setting whose quantitative behaviour cannot be extracted from the paper's results alone. Anyone reusing this pipeline on a new problem must re-tune α and should report the forward C_d at that α against the hard-mask value to understand how much physics the smoothing has removed.

5 Implementation

The solver is implemented in Python using JAX [16]. Every function is pure and compatible with `jax.jit`. The simulation state is a NamedTuple containing the distribution array $f \in \mathbb{R}^{19\times N_x\times N_y\times N_z}$, the solid mask, and the Bouzidi q buffer.

Streaming uses `jnp.roll` for periodic shifting and `jnp.where` for Bouzidi interpolation. The MRT collision uses `jax.vmap` over spatial cells with sparse moment transforms. The simulation loop compiles via `jax.lax.scan` into a single XLA program.

On an NVIDIA A100, a single BGK step on a 128^3 grid takes approximately 15 ms. MRT takes approximately 40 ms due to the per-cell moment transforms. With `jax.checkpoint` enabled, the backward pass recomputes each forward step rather than storing intermediates, adding roughly 2× wall-clock overhead. For a 200-step forward simulation, the combined forward+backward pass takes approximately 6 s for BGK and 16 s for MRT. Without checkpointing, memory scales as $O(N)$ and exceeds A100 capacity (40 GB) beyond roughly 50 steps on a 128^3 grid.

The code is open-source at `https://github.com/MarcosAsh/LBM_JAX_Autodiff` and includes 49 automated tests and a Modal cloud GPU worker for reproducibility.

All numerical experiments in this paper were run with JAX and jaxlib $\geq$ 0.4.20 (observed 0.9.2) and optax $\geq$ 0.1.7 on a single NVIDIA A100 40 GB via Modal. No pseudo-random seeds are used: the simulation is deterministic given the geometry, boundary conditions, and step count, so all tables are reproducible bit-for-bit from the released commit. The `modal_worker.py` driver provides named entry points for every result reported below (e.g. `--example cd-convergence`, `--example grad-convergence`, `--example optimisation-v2`).

6 Conclusion

We have presented a differentiable D3Q19 Lattice Boltzmann solver with second-order Bouzidi interpolated bounce-back boundary conditions whose fractional wall distances q are differentiable with respect to geometry via ray-surface intersection. The solver is validated against the Poiseuille analytic solution (L2 error 0.38%) and against the Clift, Grace, and Weber drag correlation at $Re = 100$ through a four-resolution grid convergence study with time-averaged C_d, with the finest grid settling near 7% relative error where the remaining gap is consistent with the blockage correction and un-tuned MRT damping. Local gradient correctness is verified against finite differences in float64 to relative errors below 10^{-9}.

The enabling observation is that in the classical LBM formulation (halfway bounce-back with a binary solid mask) the drag is a piecewise-constant function of smooth shape parameters, so the AD gradient is *exactly zero* and gradient-based shape optimisation is impossible in that setting. Our pipeline closes this gap: replacing $q = 0.5$ with the ray-intersection $q(r)$ yields a non-zero gradient that Adam uses to reduce a physically plausible baseline $C_d = 1.77$ to 1.66 in 60 iterations by shrinking the radius. Adding a sigmoid-smoothed solid representation supplies a second gradient path through the collision step; we find that this path dominates the numerical magnitude of the gradient when it is present (so "stronger-gradient" claims that compare soft-porosity variants are really claims about the porosity path, not about Bouzidi), and that objectives normalised by projected area admit degenerate optima that grow the smoothed obstacle to plug the channel. These are design considerations, not solver defects, and the paper documents them so that downstream users can avoid them.

Limitations and future work

The current demonstration optimises a single scalar parameter (sphere radius). Extending to higher-dimensional design spaces – multiple control points on a spline surface, or free-form deformation fields – introduces challenges that this work does not address. The Bouzidi q values can become discontinuous when a lattice link transitions between hitting and missing the obstacle surface, creating a piecewise-smooth loss landscape. For complex shapes, the number of such discontinuities grows with the surface area and grid resolution. Whether Adam or other first-order optimisers can navigate this landscape reliably at scale remains an open question.

The gradient convergence study in Section 3.3.2 also makes visible two practical limits of the current pipeline. First, Bouzidi is no longer accurate when the sphere is resolved with fewer than roughly six cells across, so the optimiser cannot be allowed to shrink the radius into that regime; we enforce this with a lower radius bound. Second, at the long-horizon gradients the optimisation relies on, AD and FD disagree by 10–40% even on the correctly-signed cases, a noise band that is consistent with float32 precision, unsteady wake sampling, and the sigmoid-smoothed geometry. The optimisation in Section 4 is robust to this level of gradient noise because Adam averages it across iterations, but applications that need a calibrated gradient (e.g. second-order methods) would not be served by this setup without further work.

Planned extensions include differentiable ray-triangle intersection for mesh-based geometries (removing the restriction to analytic shapes), higher-dimensional design spaces such as free-form deformation fields, and application to more complex bodies such as Ahmed body drag reduction at higher Reynolds numbers.

Data availability

Source code, tests, and examples: `https://github.com/MarcosAsh/LBM_JAX_Autodiff`.

References

[1] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E.M. Viggen, *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2017.

[2] M.M. Tekitek, M. Bouzidi, F. Dubois, and P. Lallemand, "Adjoint lattice Boltzmann equation for parameter identification," *Computers & Fluids*, vol. 35, pp. 805–813, 2006.

[3] M.J. Krause, G. Thäter, and V. Heuveline, "Adjoint-based fluid flow control and optimisation with lattice Boltzmann methods," *Computers & Mathematics with Applications*, vol. 65, pp. 945–960, 2013.

[4] S. Norgaard, O. Sigmund, and B. Lazarov, "Topology optimization of unsteady flow problems using the lattice Boltzmann method," *Journal of Computational Physics*, vol. 307, pp. 291–307, 2016.

[5] P. Holl, V. Koltun, and N. Thuerey, "Learning to control PDEs with differentiable physics," in *International Conference on Learning Representations (ICLR)*, 2020.

[6] M. Bedrunka, D. Wilde, M. Kliemank, D. Reith, H. Foysi, and A. Krämer, "Lettuce: PyTorch-based lattice Boltzmann framework," in *Computational Science – ICCS 2021*, Lecture Notes in Computer Science, Springer, 2021, pp. 40–55.

[7] M. Ataei, H. Salehipour, and D. Brinkerhoff, "XLB: A differentiable massively parallel lattice Boltzmann library in Python," *arXiv:2311.16080*, 2024.

[8] J. Latt, C. Malaspinas, D. Kontaxakis, A. Parmigiani, D. Lagrava, F. Brogi, M.B. Belgacem, Y. Thorimbert, S. Leclaire, S. Li, F. Marson, J. Lemus, C. Kotsalos, R. Conber, and B. Chopard, "Palabos: Parallel Lattice Boltzmann Solver," *Computers & Mathematics with Applications*, vol. 81, pp. 334–350, 2021.

[9] M. Bouzidi, M. Firdaouss, and P. Lallemand, "Momentum transfer of a Boltzmann-lattice fluid with boundaries," *Physics of Fluids*, vol. 13, no. 11, pp. 3452–3459, 2001.

[10] P.L. Bhatnagar, E.P. Gross, and M. Krook, "A Model for Collision Processes in Gases," *Physical Review*, vol. 94, no. 3, pp. 511–525, 1954.

[11] D. d'Humières, I. Ginzburg, M. Krafczyk, P. Lallemand, and L.S. Luo, "Multiple-relaxation-time lattice Boltzmann models in three dimensions," *Phil. Trans. R. Soc. A*, vol. 360, pp. 437–451, 2002.

[12] R. Clift, J.R. Grace, and M.E. Weber, *Bubbles, Drops, and Particles*, Academic Press, 1978.

[13] R. Mei, L.S. Luo, and W. Shyy, "Force evaluation in the lattice Boltzmann method involving curved geometry," *Physical Review E*, vol. 65, 041203, 2002.

[14] Q. Zou and X. He, "On pressure and velocity boundary conditions for the lattice Boltzmann BGK model," *Physics of Fluids*, vol. 9, no. 6, pp. 1591–1598, 1997.

[15] B. List, L. Chen, and N. Thuerey, "Learned Turbulence Modelling with Differentiable Fluid Solvers," *NeurIPS Workshop on Machine Learning and the Physical Sciences*, 2022.

[16] J. Bradbury et al., "JAX: composable transformations of Python+NumPy programs," 2018, `http://github.com/jax-ml/jax`.